Internship Application Copilot — Final Implementation Plan (v3.0)

Status: This is the single source of truth. Supersedes v2.0 and the standalone v3.0 addendum — do not read those alongside this one.

Read this before implementing anything:

1. Everything under §§1–16 is fully specified and ready to build.
2. §17 lists three features that are **intentionally not designed yet**. Do not implement them, do not infer a design for them, and do not touch code paths related to them beyond what's already needed for §§1–16. If you (the coding agent) think a task requires deciding how one of those should work, stop and ask rather than guessing.
3. If anything in this document is ambiguous or looks contradictory, ask before proceeding rather than resolving it silently — a wrong guess here fills real people's job applications with wrong data.

---

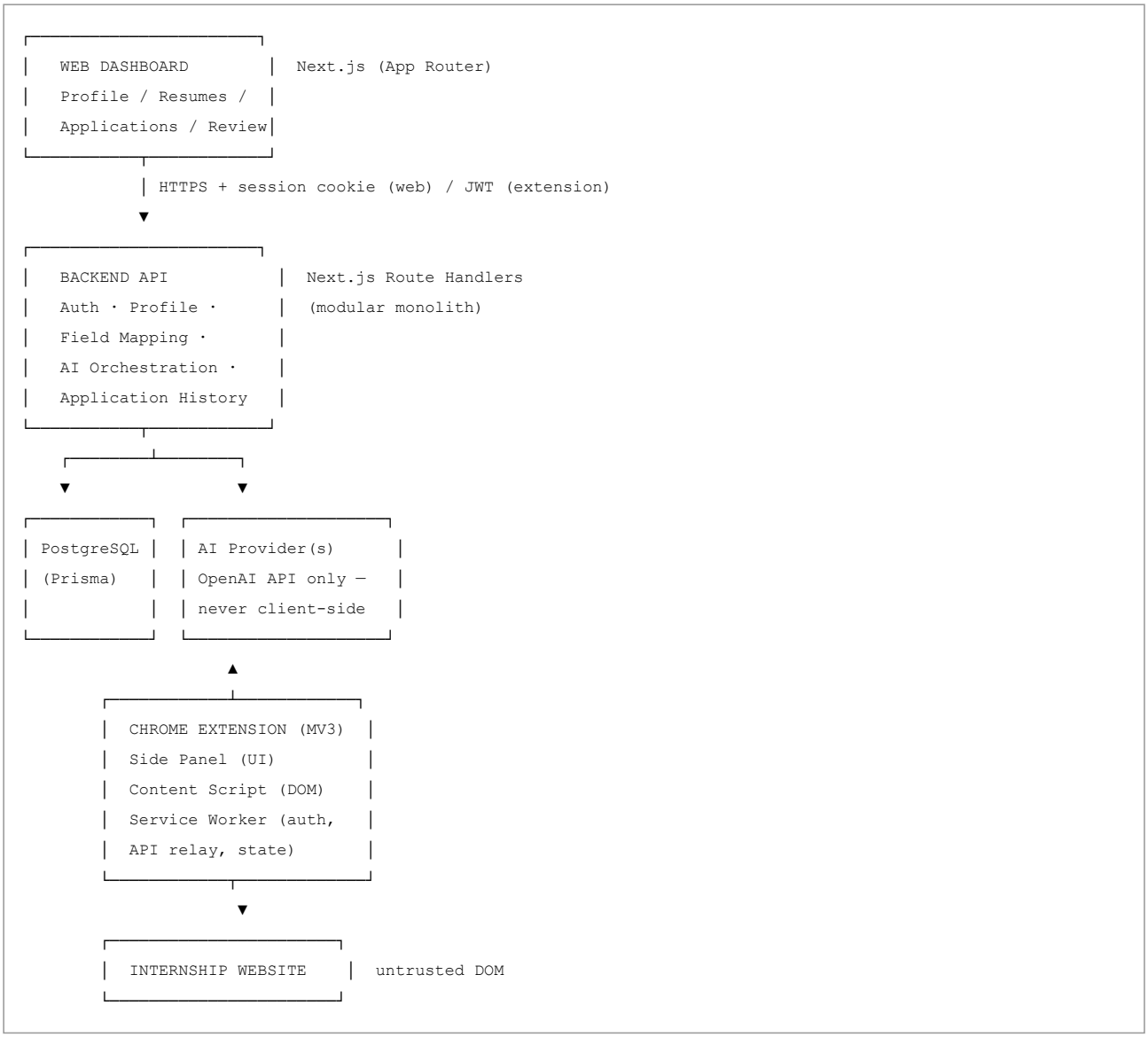
0. Summary of What Changed From v2.0

#	Area	v2.0	v3.0	Why
13	Fill execution model	Fields classified, then filled — implicitly all at once, no verification step	Fields filled **one at a time, in DOM order**, each verified against a type-aware read-back before the queue advances to the next field	Batch filling had no checkpoint to catch event-dispatch failures until final review; custom components (React Select, date pickers) need a real click sequence, not just a value set; concurrent fills raced with MutationObserver-triggered re-scans
14	Fill UX	None specified	Optional animated cursor overlay showing which field is being filled, purely cosmetic, toggle-able, proven not to affect fill results (§6.6)	User-facing trust/transparency for an extension typing into someone's job application — separate from the accuracy fix in #13, which comes from verification, not visuals

Everything from v2.0 not mentioned above (architecture, tech stack, DB schema, API contracts, MV3 constraints, security/privacy, error handling baseline, monitoring, known limitations, phasing) is retained as-is and reproduced below with the two changes above woven in at their relevant sections (§2, §6.6, §8, §11, §12, §13, §15).

---

1. Architecture



Decision → Reason → Trade-off: Backend stays a modular monolith inside the same Next.js app, not a separate service. → Single team, single deploy target; the real risk is the field-mapping/fill engine, not service-to-service scaling. → If AI orchestration load grows heavily, routes will need extraction into a worker/queue later; keep `packages/ai` cleanly separated from HTTP handlers now so that's a lift-and-shift, not a rewrite.

---

2. Component Responsibilities (Rulebook)

1. Extension owns browser interaction. It never calls the AI provider directly — only the backend does.
2. Web app owns profile management — single source of truth for profile CRUD.
3. Backend owns all business logic, including confidence-threshold decisions. The extension trusts the backend's `action` field but independently re-validates it against a local Zod schema before touching the DOM (defense in depth).
4. PostgreSQL is the source of truth for persisted profile/application data. Chrome storage is a cache only (session token, last-used profile snapshot, in-progress fill session state) and must be safe to lose.
5. The AI never directly controls the browser. LLM output is JSON → Zod-validated → business-rule-checked → only then does the autofill engine touch the DOM.
6. Every AI output is schema-validated before use, no exceptions, including confidence score bounds (0–1) and enum-constrained action.
7. Deterministic rules run before AI, always, per field.
8. Low-confidence mappings are never silently filled — surfaced to user as "needs review," never guessed.
9. User must review and explicitly approve any AI-drafted subjective answer before it enters a form field.
10. No automatic submission, ever.
11. Site-specific adapters are opt-in overrides, not the primary path (§9.5 equivalent, retained from v2.0, not repeated here since unchanged).
12. Every production bug becomes a fixture + regression test.

13. **NEW — The autofill engine fills fields sequentially, one at a time, and verifies each fill before advancing.** It never dispatches fills to multiple fields concurrently. A field is only marked `filled` after its resulting DOM state has been read back and matches the intended value.

---

3. Tech Stack

Layer	Choice	Reason
Web frontend	Next.js (App Router), TypeScript, Tailwind, shadcn/ui, React Hook Form, Zod	Zod schemas shared with backend and extension via monorepo package
Extension	Chrome MV3, TypeScript, React, Vite, `chrome.sidePanel`, `chrome.scripting`, `chrome.storage.session`	Side Panel is the supported MV3 surface for persistent UI; Vite gives fast HMR
Backend	Next.js Route Handlers + Prisma + PostgreSQL, modular monolith	Single deploy, shared types, sufficient for MVP scale
AI provider	OpenAI API, tiered model routing	Structured outputs suit field-classification exactly; abstracted behind `packages/ai/client.ts`
Object storage	S3-compatible (AWS S3 / Cloudflare R2)	Documents don't belong in Postgres
Auth	Auth.js (web), `chrome.identity` + backend-issued short-lived JWT (extension)	Single account across surfaces
Monorepo	pnpm + Turborepo	Shared `packages/types`, `packages/validation`, `packages/ai`

---

4. Folder Structure

```
internship-copilot/
├── apps/
│   ├── web/                              # Next.js dashboard
│   └── extension/
│       ├── src/
│       │   ├── background/
│       │   │   └── service-worker.ts     # stateless-safe; rehydrates from chrome.storage.session
│       │   ├── content/
│       │   │   ├── content-script.ts
│       │   │   ├── dom-scanner.ts
│       │   │   ├── shadow-dom-walker.ts
│       │   │   ├── frame-registry.ts
│       │   │   ├── field-detector.ts
│       │   │   ├── field-mapper.ts
│       │   │   ├── fill-queue.ts         # NEW — sequential fill state machine, §6
│       │   │   ├── cursor-overlay.ts     # NEW — cosmetic animation, §6.6
│       │   │   ├── event-dispatcher.ts   # fires React/Vue-compatible synthetic events
│       │   │   ├── form-validator.ts
│       │   │   └── mutation-observer.ts
│       │   ├── sidepanel/
│       │   ├── storage/
│       │   │   └── chrome-storage.ts     # session-scoped token + fill-progress storage
│       │   └── types/
│       ├── manifest.json
│       └── vite.config.ts
├── packages/
│   ├── database/
│   ├── types/
│   ├── validation/
│   ├── ai/
│   │   ├── client.ts
│   │   ├── prompts/
│   │   ├── field-mapper.ts
│   │   ├── job-analyzer.ts
│   │   ├── answer-generator.ts
│   │   └── resume-matcher.ts
│   └── config/
├── tests/
│   ├── fixtures/forms/
│   │   ├── iframe-embedded.html
│   │   ├── shadow-dom.html
│   │   ├── csp-restricted.html
│   │   ├── react-controlled-inputs.html
│   │   ├── sequential-custom-dropdown.html      # NEW
│   │   ├── sequential-debounced-validation.html # NEW
│   │   ├── sequential-dynamic-reflow.html       # NEW
│   │   └── cursor-overlay-parity.html           # NEW
└── ...
```

5. Database Schema

(Unchanged from v2.0 — reproduced for completeness since this doc is the sole source of truth.)

```
User
  id, email, authProvider, createdAt

Profile
  id, userId, personal(json or normalized table), links(json)

Education / Experience / Project / Skill / Achievement / Certification
  — each FK'd to Profile

Document
  id, userId, type(resume|cover_letter|transcript|certificate|portfolio),
  filename, mimeType, storageKey, size, tags[], createdAt

Application
  id, userId, url, domain, jobTitle, status, sessionId, createdAt, updatedAt

ApplicationField
  id, applicationId, rawLabel, normalizedLabel, domSelectorHash,
  profilePath, confidence, action(fill|review|skip),
  source(rule|ai_fast|ai_strong|user_override), finalValue, createdAt

FieldMapping (user-specific learned corrections)
  id, userId, normalizedLabel, domainScope(nullable), profilePath,
  confidence, createdAt

AIInteraction
  id, userId, applicationId, operation(classify|job_analysis|answer_gen|resume_match),
  inputTokens, outputTokens, model, latencyMs, success, errorCode, createdAt
  — never store raw webpage HTML/text sent to the model; field labels + result only

UserPreference
  id, userId, autofillConfidenceThreshold, preferredResumeId, notificationSettings,
  showFillCursor(boolean, default true)   # NEW — user toggle for §6.6

AuditLog
  id, userId, actorType(user|system|ai), action, targetType, targetId, createdAt
```

---

6. Sequential Fill Engine (the core v3.0 change)

6.1 Why sequential-and-verified, not batch

Batch filling had four concrete failure modes:

1. Silent event-dispatch failures were only discoverable at final review, after every field had already been attempted.
2. Custom components (React Select, MUI Autocomplete, date pickers) often only initialize/open on a real `pointerdown → focus → click` sequence, not a value-set alone.
3. Concurrent fills raced with `MutationObserver`-triggered re-scans for dynamic forms (Phase 5).
4. No natural checkpoint existed to respect debounced validators (e.g., an email format check that fires 300ms after blur).

6.2 Pipeline order (amends v2.0 §9's detection pipeline)

```
DOM Field → Normalize → Rule Matcher → Confidence Score
  → if ≥0.95: queue for sequential fill
  → if 0.80–0.94: queue for sequential fill + flag for review
  → if 0.50–0.79: ask user (do not queue)
  → if <0.50: do not queue, mark unsupported

[all fields classified first, exactly as before]
        ↓
Sequential Fill Queue (NEW — replaces "fill" step)
  → ordered by DOM position (document order / tabIndex)
  → one field at a time, verified before advancing
```

Classification stays batched (cheap, no DOM side effects, fine to parallelize/cache per §10.3 of v2.0). Only the **filling** step — which touches the live page — becomes sequential.

6.3 `fill-queue.ts`

```
type QueuedField = {
  frameId: number;
  domSelectorHash: string;
  element: HTMLElement; // resolved at dequeue time, not held long-term
  profilePath: string;
  value: string;
  fieldType: "text" | "select" | "custom-dropdown" | "checkbox" | "radio" | "date";
};

type FillResult = {
  fieldId: string;
  status: "filled" | "review" | "skipped" | "failed";
  attempts: number;
};

async function runFillQueue(
  queue: QueuedField[],
  opts: { interFieldDelayMs: number; showCursor: boolean }
): Promise<FillResult[]> {
  const results: FillResult[] = [];
  for (const field of queue) {
    const result = await fillOneField(field, opts);
    results.push(result);
    persistProgress(results); // -> chrome.storage.session, §6.5
    await delay(opts.interFieldDelayMs);
  }
  return results;
}
```

6.4 `fillOneField` — per-field state machine

```
async function fillOneField(
  field: QueuedField,
  opts: { showCursor: boolean }
): Promise<FillResult> {
  const el = field.element;

  el.scrollIntoView({ behavior: "smooth", block: "center" });
  await waitForInView(el, { timeoutMs: 800 });

  if (opts.showCursor) {
    await animateCursorTo(el); // cosmetic only, §6.6
  }

  dispatchPointerSequence(el); // pointerdown -> mousedown -> focus -> pointerup -> mouseup -> click
  const gotFocus = await waitForFocus(el, { timeoutMs: 300 });
  if (!gotFocus) {
    return { fieldId: field.domSelectorHash, status: "review", attempts: 1 };
  }

  setNativeValueAndDispatch(el, field.value); // existing event-dispatcher.ts logic
  dispatchBlur(el);

  const verified = await verifyFieldValue(el, field.value, field.fieldType, { timeoutMs: 400 });
  if (verified) {
    return { fieldId: field.domSelectorHash, status: "filled", attempts: 1 };
  }

  // One retry with a longer settle window — covers slow framework re-renders.
  await delay(300);
  const verifiedRetry = await verifyFieldValue(el, field.value, field.fieldType, { timeoutMs: 600 });

  return {
    fieldId: field.domSelectorHash,
    status: verifiedRetry ? "filled" : "review",
    attempts: 2,
  };
}
```

`verifyFieldValue` is type-aware:

- text/date inputs → read `.value`
- native `<select>` → read `.selectedOptions`
- custom dropdown → read rendered display text / `aria-selected` on the option (no underlying `<select>` to read)
- checkbox/radio → read `.checked`

This is what actually closes v2.0's §15 known limitation ("shows filled visually but form state doesn't register") — sequential-with-verify catches it per-field instead of only at final review.

6.5 Progress persistence (amends v2.0 §8.2 — MV3 service worker lifecycle)

Service workers can die mid-queue (Chrome kills them after ~30s idle). Progress must survive a respawn:

```
// chrome.storage.session, keyed per sessionId
type FillProgress = {
  sessionId: string;
  queueOrder: string[]; // domSelectorHash[], committed order
  results: FillResult[]; // appended after every field, not batched
  currentIndex: number;
};
```

On respawn mid-fill, the side panel reads `FillProgress` and either resumes from `currentIndex`, or — if the tab was reloaded and the DOM no longer matches — re-runs field detection and reconciles by `domSelectorHash` before resuming. It never blindly restarts from field 0.

6.6 Cursor overlay — cosmetic layer, explicitly non-authoritative

```
// Shadow DOM host — isolated from page CSS/JS in both directions.
function animateCursorTo(el: HTMLElement): Promise<void> {
  const rect = el.getBoundingClientRect();
  return cursorOverlay.moveTo(rect.left, rect.top, { durationMs: 250 });
}
```

- User-facing toggle in the side panel (`UserPreference.showFillCursor`, default on).
- **Hard rule:** `showCursor: false` must produce bit-identical `FillResult[]` to `showCursor: true`. Enforced by running the fixture suite twice (once per flag) and asserting identical output — see `cursor-overlay-parity.html` in §4. If these ever diverge, that's a bug in the queue logic leaking into the cosmetic layer, not an acceptable trade-off.

6.7 Trade-off, stated plainly

Decision: Fill sequentially with per-field verification instead of batch. **Reason:** Catches event-dispatch and custom-component failures per-field instead of only at final review; eliminates races with concurrent DOM mutation. **Trade-off:** Fill time scales linearly with field count instead of being parallelizable — a 20-field form takes on the order of a few seconds longer, dominated by the 300–600ms verify-with-retry windows (the cosmetic cursor animation adds a smaller, separately-disableable amount on top). For MVP scale — one form at a time, human review after — this is an acceptable and arguably desirable trade: a visibly-progressing, verified fill is more trustworthy to an applicant than an instant one that might silently be wrong.

---

7. MV3 Extension Architecture

- **Permissions:** `activeTab`, `storage`, `sidePanel`, `scripting` at install. Site-specific host permission requested via `chrome.permissions.request()` only when the user clicks "Start Autofill" on that page — never `<all_urls>` upfront.
- **Service worker:** non-persistent; all session state (auth token, sessionId, fill progress per §6.5) lives in `chrome.storage.session`, never assumed to survive in memory between messages.
- **Content script injection:** `allFrames: true` to reach same-origin iframes; every detected field carries a `frameId` so fills route back to the correct frame via `chrome.scripting.executeScript({ frameIds: [...] })`.
- **Cross-origin iframes:** hard limitation, not a bug to fix. Cannot read a cross-origin iframe's DOM from the parent frame's JS context (same-origin policy). Coordination happens via `postMessage` between injected content-script instances, relayed through the service worker, when both origins are covered by `matches` + `all_frames: true`.
- **Shadow DOM:** scanner recursively pierces `shadowRoot` when `mode: "open"`. Closed shadow roots are unreachable by design — surfaced in the review UI as "unsupported field," never silently skipped.
- **Framework-controlled inputs:** values set via the native input value setter, with a real `InputEvent`/`Event('input', {bubbles:true})` and `change` dispatched — not a bare `.value` assignment, which React's controlled-component state won't register.
- **CSP:** content scripts run in an isolated world, so standard DOM APIs work under restrictive CSP. Never inject a `<script>` tag into the page to help with autofill, under any circumstance.

---

8. Form Detection + Field-Mapping Pipeline

See §6.2 for the full pipeline including the sequential fill step. The classification tiers themselves are unchanged from v2.0:

```
DOM Field → Normalize → Rule Matcher → Confidence Score
  → ≥0.95: auto-fill (via sequential queue)
  → 0.80–0.94: auto-fill + flag for review
  → 0.50–0.79: ask user, do not fill
  → <0.50: do not fill, mark unsupported
```

Site-specific adapter overrides remain opt-in only, config-driven, code-reviewed, and require a linked fixture:

```
type AdapterOverride = {
  domainPattern: string;
  reason: string; // required — documents WHY generic detection failed
  fieldOverrides?: Record<string, string>;
  frameStrategy?: "pierce-shadow" | "cross-origin-postmessage";
};
```

---

9. AI Architecture + Model Routing

- **Rule engine (no AI):** email, phone, name, GitHub/LinkedIn/portfolio URLs, known education fields.
- **Fast/cheap tier:** simple field classification, text normalization, basic categorization.
- **Workhorse tier:** ambiguous field interpretation, job description structuring, subjective answer drafting, resume-to-JD matching.
- **Top reasoning tier — opt-in only:** reserved for cases where the workhorse tier's confidence is still low after one retry, and only if the user enables "high-effort mode." Not routed to by default.
- **Structured output enforcement:** every model call uses JSON-schema-constrained output; every response passes through the same Zod schema used for rule-engine outputs, so downstream logic is model-agnostic.
- **Cost controls:** cache classification by normalized label (not literal DOM field); batch ambiguous fields from the same page into one call; track `AIInteraction` tokens/model/latency per call.

Data minimization (enforced at the type level, not just convention):

```
function classifyField(
  field: { label: string; type: string; name?: string; nearbyText?: string },
  candidateProfilePaths: string[]
): Promise<FieldClassification>
// Cannot accept a full Profile object.
```

---

10. Security & Privacy

- **Prompt injection:** webpage text is untrusted input. Never passed as instructions — always wrapped as clearly delimited data. Model output is constrained to a fixed enum of `profilePath` values via structured outputs; no `"submit"` action exists in the schema at all. The backend re-validates every AI-suggested `profilePath` against the actual candidate list; anything outside it is rejected regardless of what the model returned.
- **Extension ↔ backend auth:** `chrome.identity` OAuth token exchanged once for a 15-minute backend JWT at `POST /api/auth/extension-token`. Stored in `chrome.storage.session` only, never `.local`. Refresh token rotates on use; both tokens revocable server-side per-device. Backend independently validates the user owns any `sessionId`/`applicationId` referenced — never trusts a client-supplied `userId`.
- **Retained:** never store passwords/OTP/bank/card data; encrypt sensitive data at rest; never send full profile to AI; documents validated by type + size limit; parameterized queries via Prisma; schema'd audit logs.

---

11. API Contracts

```
POST /api/auth/extension-token
  Request:  { chromeIdentityToken: string }
  Response: { accessToken: string, expiresIn: 900, refreshToken: string }

POST /api/autofill/session
  Request:  { url, domain, title }
  Response: { sessionId }

POST /api/autofill/analyze
  Request:  { sessionId, fields: FieldDescriptor[] }
  Response: { mappings: FieldMapping[] }

POST /api/autofill/complete
  Request:  { sessionId, filledFieldIds: string[], skippedFieldIds: string[], errors: FieldError[] }
  Response: { applicationId, status }

POST /api/ai/analyze-job
POST /api/ai/generate-answer
  Request:  { questionText, questionCategory, relevantProfileFields: string[] }
  Response: { draftAnswer, confidence }
POST /api/ai/match-resume

GET  /api/applications
POST /api/applications
PATCH /api/applications/:id
```

All extension-originating write requests require an `Idempotency-Key` header (client-generated UUID per action), so retry-after-timeout on `/api/autofill/complete` can't create duplicate `Application` rows.

---

12. Error Handling

Failure	Handling
AI call timeout/5xx	Retry once with backoff (1s, 3s); on second failure, fall back to rule-engine-only result, `source: "rule_fallback"`, never blocks the session
AI returns schema-invalid JSON	Zod fails closed → field marked `action: "review"`, never auto-filled, logged to `AIInteraction.errorCode`
Extension loses connection to service worker mid-fill	Side panel detects port disconnect, re-establishes, re-requests current session state from backend (backend, not memory, is authoritative)
Duplicate `/api/autofill/complete` due to retry	Deduplicated via `Idempotency-Key`
NEW — field fill dispatched but read-back value doesn't match after retry window	Marked `action: "review"`, never counted as `"filled"` in the completion summary; logged with `attempts: 2` so §14 metrics distinguish "never attempted" from "attempted and failed to register"
NEW — custom dropdown option not found by display text	Falls back to `review` immediately, no retry — a failed option match rarely changes on a second attempt
Rate limit hit on AI provider	Queue remaining classifications, surface "still analyzing…" in the side panel rather than failing the whole page scan

---

13. Testing

```
tests/fixtures/forms/
  iframe-same-origin.html
  iframe-cross-origin.html          # asserts graceful degradation, doesn't "pass"
  shadow-dom-open.html
  shadow-dom-closed.html            # asserts correct "unsupported" reporting
  react-controlled-inputs.html      # asserts native setter + event dispatch required
  csp-restricted.html
  prompt-injection-attempt.html     # asserts schema holds regardless of injected text
  sequential-custom-dropdown.html      # NEW — asserts click-sequence, not just focus, required
  sequential-debounced-validation.html # NEW — asserts verify-with-retry catches 300ms-debounced validators
  sequential-dynamic-reflow.html       # NEW — asserts queue re-resolves by selector hash after mid-fill DOM shift
  cursor-overlay-parity.html           # NEW — asserts showCursor true/false give identical FillResult[]
```

---

14. Monitoring & Observability

- Extension-side: structured error reporting keyed by anonymized domain hash, not full URL with query params.
- Backend-side: per-route latency + error rate; per-model latency/cost/error rate from `AIInteraction`.
- Product metric: rule-match rate vs. AI-fallback rate per domain, to catch silent deterministic-layer degradation.
- **NEW:** per-field fill outcome distribution (`filled` / `review` / `failed`) and `attempts` histogram, to catch when a specific ATS platform is systematically failing verification (signals a new adapter override may be needed, per §8).

---

15. Known Limitations (state these to users, don't hide them)

- Cross-origin iframe forms may only partially autofill; some fields require manual entry.
- Closed Shadow DOM fields cannot be detected or filled.
- Pages that block synthetic input events under strict CSP may still land in `review` state via the §6.4 verification step rather than falsely showing as filled.
- Confidence/accuracy targets are engineering targets to validate against the fixture suite and real ATS samples, not guaranteed figures.
- **NEW:** Sequential fill is slower than a hypothetical instant batch fill by design — expect a few seconds of visible per-field progress on longer forms. This is a stated trade-off (§6.7), not a bug.

---

16. Development Phases — Acceptance Criteria

Phases 0–8 retained in order (profile schema → detection engine → deterministic matcher → basic extension autofill → dashboard → backend wiring → AI mapping → dynamic/multi-step → resume intelligence → tracking).

- **Phase 2 (Basic Extension)** done when: scanner correctly enumerates all fields, including same-origin iframe fields with correct `frameId`, on at least 5 distinct real ATS fixture pages.
- **Phase 3 (Deterministic Autofill) — REVISED:** done when fields are filled **sequentially** via the §6 queue, each fill verified against its type-appropriate read-back before the queue advances, `react-controlled-inputs.html` **and** `sequential-custom-dropdown.html` both pass, and a forced mid-queue service-worker termination (simulating MV3's ~30s idle kill) resumes correctly from `chrome.storage.session` without re-filling already-verified fields.
- **Phase 4 (AI Field Mapping)** done when: `prompt-injection-attempt.html` passes and cost-per-field-classification is logged and under an agreed budget.
- **Phase 5 (Dynamic Applications)** done when: `MutationObserver`-triggered rescans correctly re-run the rule→AI pipeline on newly appeared fields without duplicate submissions of already-filled ones, and `sequential-dynamic-reflow.html` passes.

---

17. Open Items — NOT specified in this document, do not implement

These were raised as goals but have no design yet. Building against them without a spec risks the agent inventing behavior the product owner hasn't approved. Flag these back for a design pass rather than proceeding:

1. **Dropdown/option field scanning bug:** dropdown fields are currently being detected as one separate field per option instead of a single field with N options. Needs a fix to `dom-scanner.ts`'s field-detection logic — not yet designed.
2. **Upload-first pipeline ordering:** resume/document upload should happen before the scanner fills remaining fields, rather than the current order. Needs a defined sequencing change to the Phase 4/6 flow — not yet designed.
3. **Resume-inference AI layer:** a new AI layer that infers profile gaps (experience, skills, expected salary/LPA) from the uploaded resume when the stored profile is missing them, cross-checked against AI-generated answers. Needs its own data-minimization treatment (§9), a new `AIInteraction.operation` value, and explicit confidence/review rules before it touches the fill pipeline — not yet designed.

Do not write code for these. If asked to "finish v3.0," implement §§1–16 first and stop at §17.